Web Server i Servleti

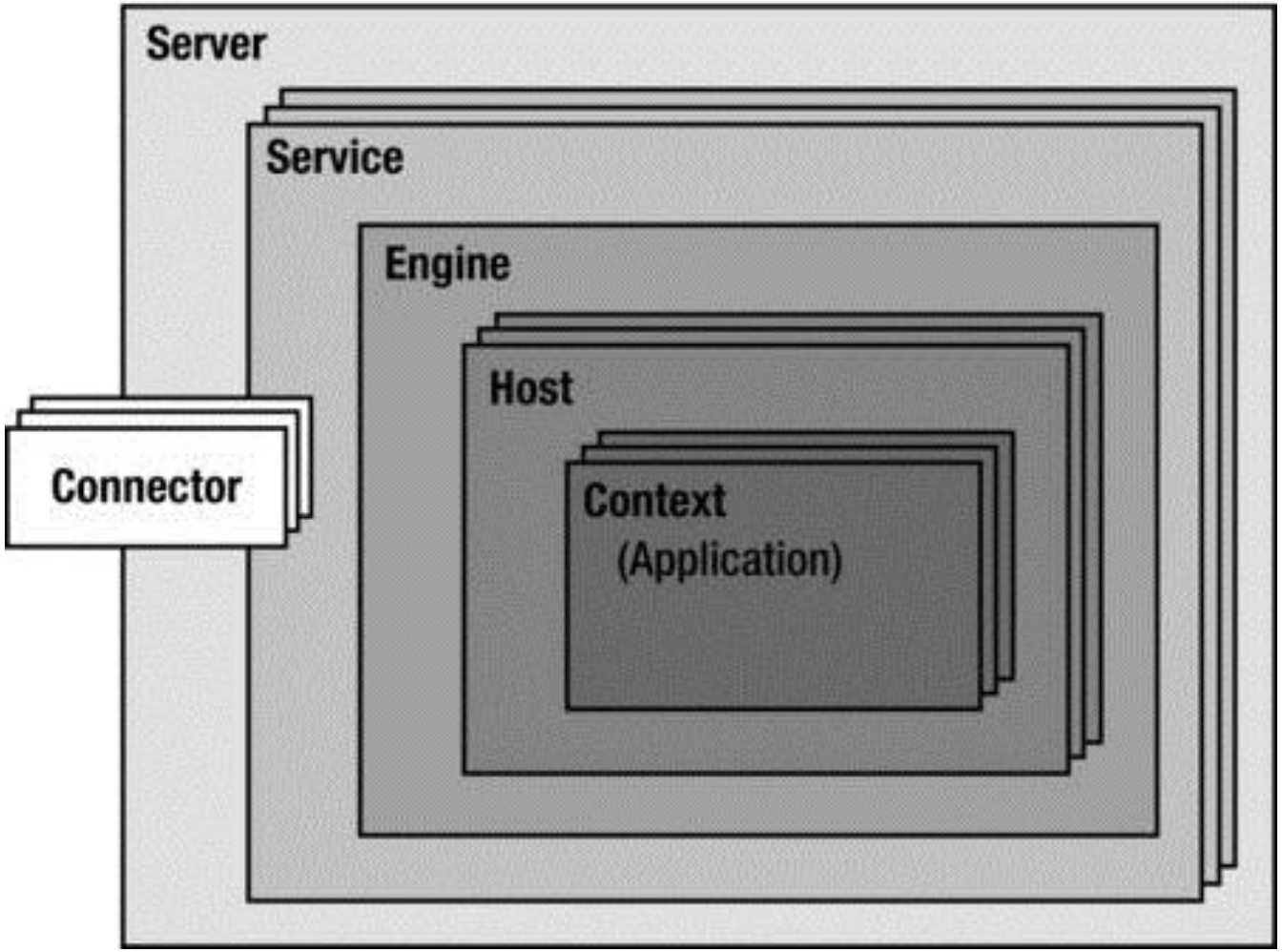
Stefan Antić

santic@raf.rs

Web Server

- Web server je softver koji služi da procesira zahteve (HTTP ili neke druge) i klijentu vrati odgovarajući odgovor (objekat JSON, XML, itd) ili web stranicu (statičku ili dinamičku).
- Statičke imaju uvek isti sadržaj i server ih vraća takve kakve jesu bez dodatnog procesiranja.
- U dinamičkim se sadržaj generiše tokom obrade zahteva.

Tomcat Web Server



Tomcat Web Server, server.xml

```xml
<?xml version='1.0' encoding='utf-8'?>
<Server port="8005" shutdown="SHUTDOWN">
   <Listener className="org.apache.catalina.core.AprLifecycleListener" SSLEngine="on" />
   <Listener className="org.apache.catalina.core.JasperListener" />
   <Listener className="org.apache.catalina.core.JreMemoryLeakPreventionListener" />
   <Listener className="org.apache.catalina.mbeans.GlobalResourcesLifecycleListener" />
   <Listener className="org.apache.catalina.core.ThreadLocalLeakPreventionListener" />
   <GlobalNamingResources>
     <Resource name="UserDatabase" auth="Container"
               type="org.apache.catalina.UserDatabase"
               description="User database that can be updated and saved"
               factory="org.apache.catalina.users.MemoryUserDatabaseFactory"
               pathname="conf/tomcat-users.xml" />
   </GlobalNamingResources>
   <Service name="Catalina">
     <Connector port="8080" protocol="HTTP/1.1"
                connectionTimeout="20000"
                redirectPort="8443" />
     <Connector port="8009" protocol="AJP/1.3" redirectPort="8443" />
     <Engine name="Catalina" defaultHost="localhost">
       <Realm className="org.apache.catalina.realm.LockOutRealm">
         <Realm className="org.apache.catalina.realm.UserDatabaseRealm"
                resourceName="UserDatabase"/>
       </Realm>
       <Host name="localhost"  appBase="webapps"
             unpackWARs="true" autoDeploy="true">
         <Valve className="org.apache.catalina.valves.AccessLogValve" directory="logs"
                prefix="localhost_access_log." suffix=".txt"
                pattern="%h %l %u %t &quot;%r&quot; %s %b" />
       </Host>
     </Engine>
   </Service>
</Server>
```

Tomcat Web Server

- Context – predstavlja jednu web aplikaciju. Konfiguriše se pomoću \WEB-INF\web.xml ili uz pomoć anotacija
- Connector – služi za komunikaciju sa klijentom, svaki zahtev se obrađuje u zasebnom thread-u, moguće je kreirati Thread Pool sa već spremnim thread-om.
- Host – domensko ime servera. Može ih biti više na istom serveru i uz pomoć njih server zna kojoj aplikaciji (context-u) treba proslediti zahtev. Npr:
  - raf.rs,
  - maillist.raf.rs,
  - servis.raf.rs
  - …

Tomcat Web Server

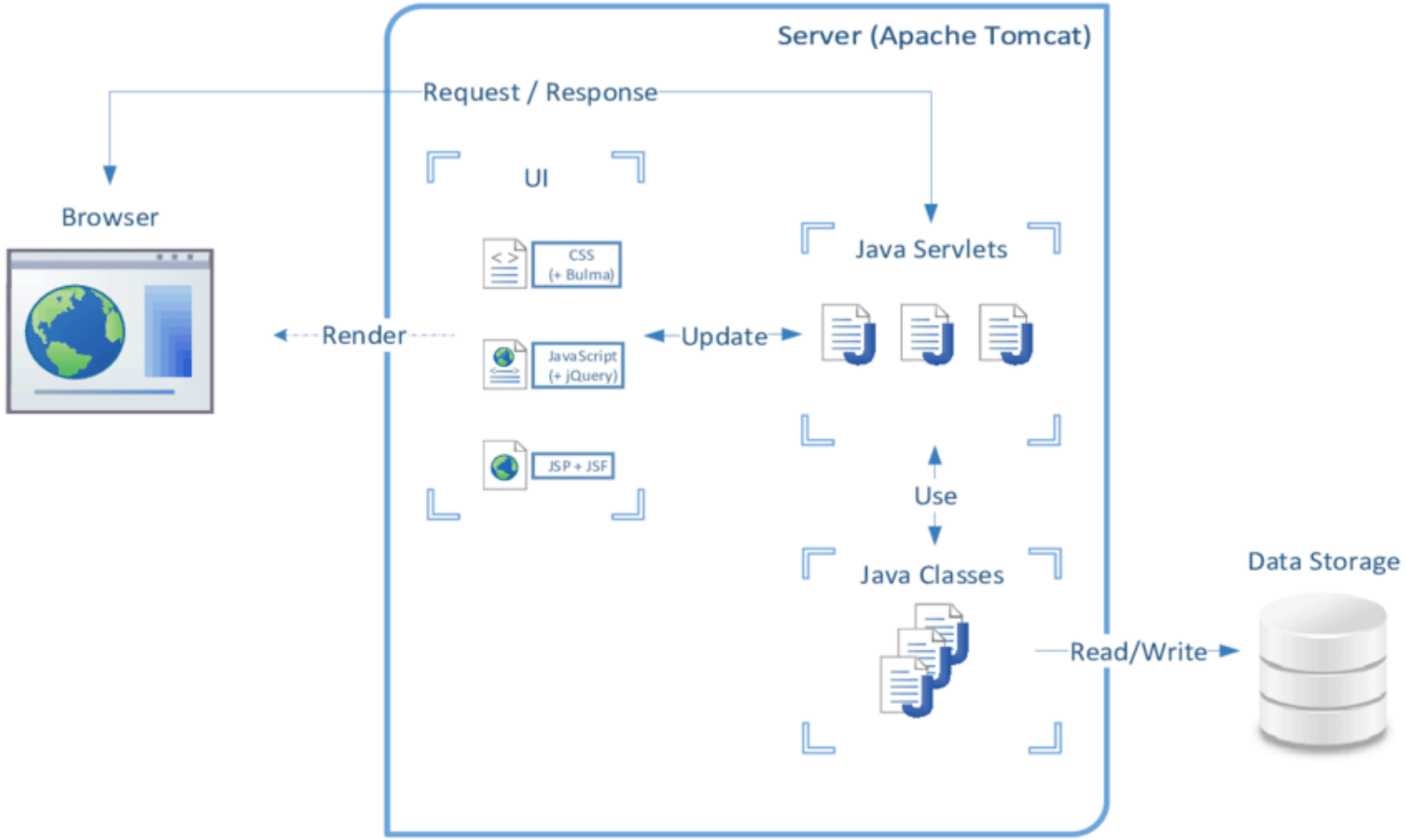
- Engine – procesira sve zahteve konektora koji su povezani sa njim.
- Service – povezuje konektore sa tačno jednim Engine-om.
- Server – predstavlja instancu Tomcat-a

- Listener – osluškuju i reaguju na događaje
- Valve – presreću HTTP zahteve

Servleti

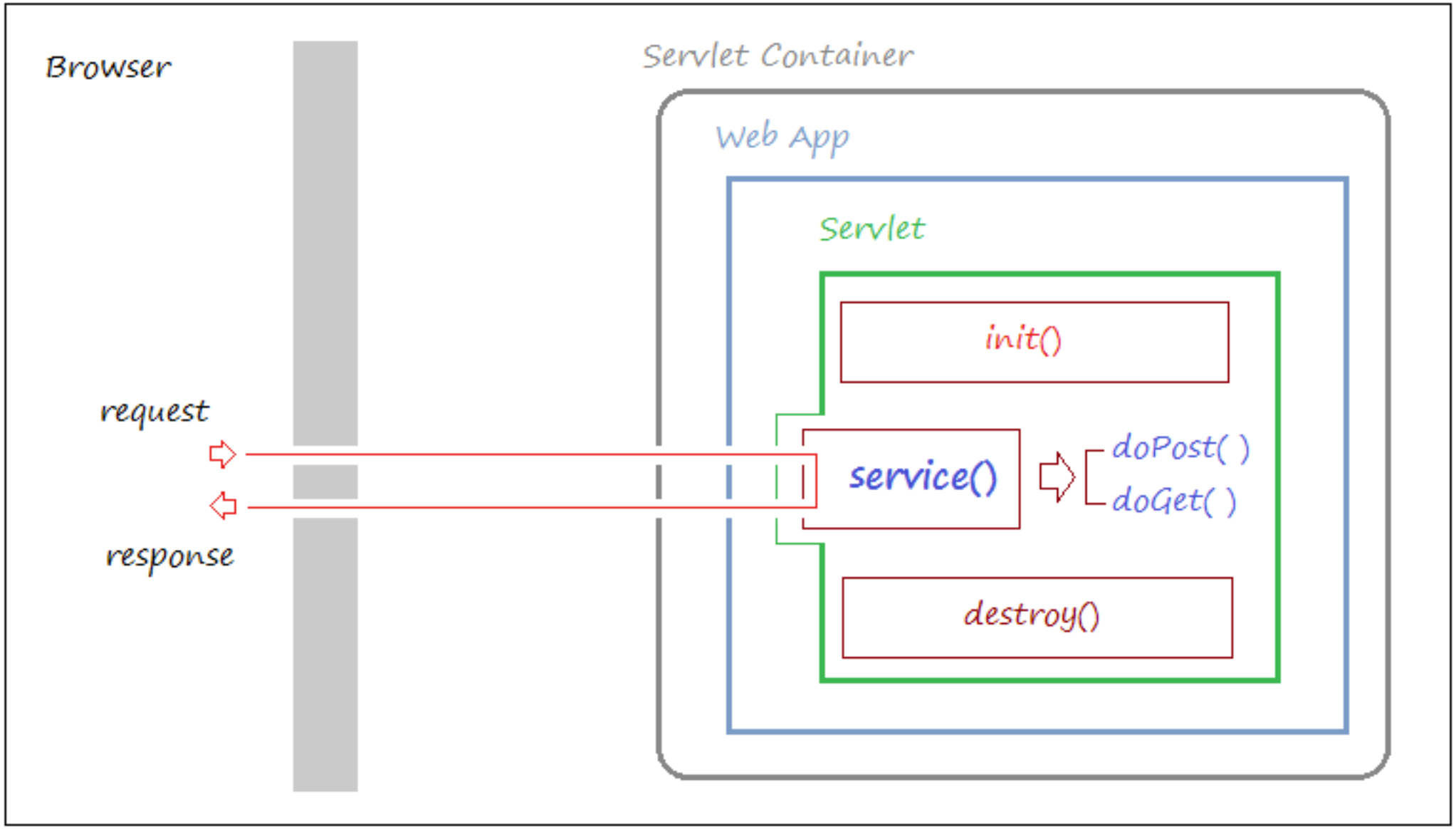
- Java klasa koja omogućava odgovaranje na zahteve.
- Klijent pošalje zahtev
- Zahtev stigne do web servera
- Web server prosledi zahtev odgovarajućem servletu
- Servlet procesira zahtev i vrati odgovarajući odgovor web serveru
- Web server vraća odgovor klijentu

Servleti


Server (Apache Tomcat)
Request / Response
Browser
UI
CSS
(+ Bulma)
JavaScript
(+ jQuery)
JSP + JSF
Render
Update
Java Servlets
Use
Java Classes
Read/Write
Data Storage

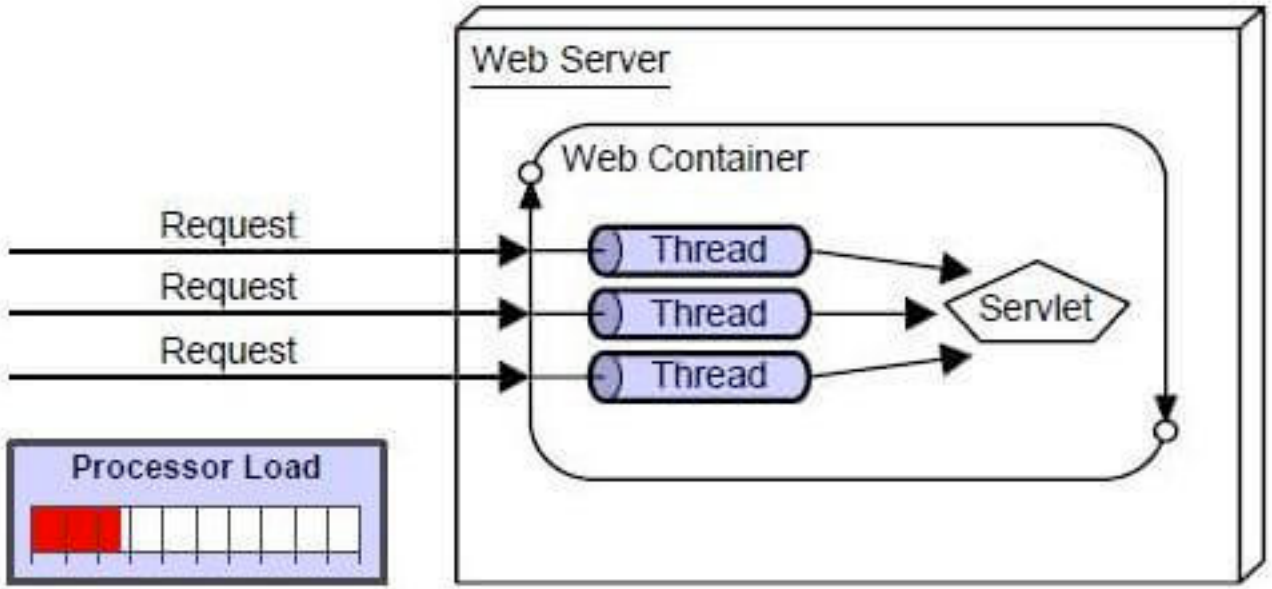
Servleti

- Servlet container – deo web servera u kome se izvršava servlet.
- Lifecycle servleta:
  - Init() – servlet container poziva ovu metodu prilikom instanciranja servleta
  - Service() – u zavisnosti od request metode, prosleđuje zahtev odgovarajućoj metodi (doGet, doPost, doPut...)
  - Destroy() – poziva se prilikom gašenja servleta (ne nakon vraćanja odgovora)


Browser
Servlet Container
Web App
Servlet
init()
request
service()
doPost( )
doGet( )
response
destroy()

Servleti

- Pri prvom zahtevu ka putanji koja je vezana za Servlet, servlet container kreira instancu tog Servleta koja je singleton.
- Svi novi zahtevi ka istoj putanji će koristiti već kreiranu instancu Servleta.
- Instanca Servleta je jedinstvena, ali se njene metode pozivaju iz različitih thread-ova.


Web Server
Web Container
Request
Request
Request
Thread
Thread
Thread
Servlet
Processor Load

Servlet

Unutar Servleta postoje metode koje se pozivaju za svaku metodu HTTP zahteva: GET, POST, PUT, DELETE…
U tim metodama pišemo logiku koja će obraditi specifičan zahtev:

doGet() – poziva se za GET zahteve

doPost() – poziva se za POST zahteve

doDelete()

doPut()

Praćenje sesije

- Servlet API omogućava praćenje sesije kroz HttpSession interfejs.
- Svaki korisnik ima svoju sesiju koja je označena nekim jedinstvenim identifikatorom: JSESSIONID
- Sesija se prati uz pomoć cookie-a.
- Servlet će za svaki zahtev proveriti da li sadrži cookie pod imenom JSESSIONID i ukoliko ne sadrži, u header-u odgovora će klijentu poslati njegov identifikator (JSESSIONID):

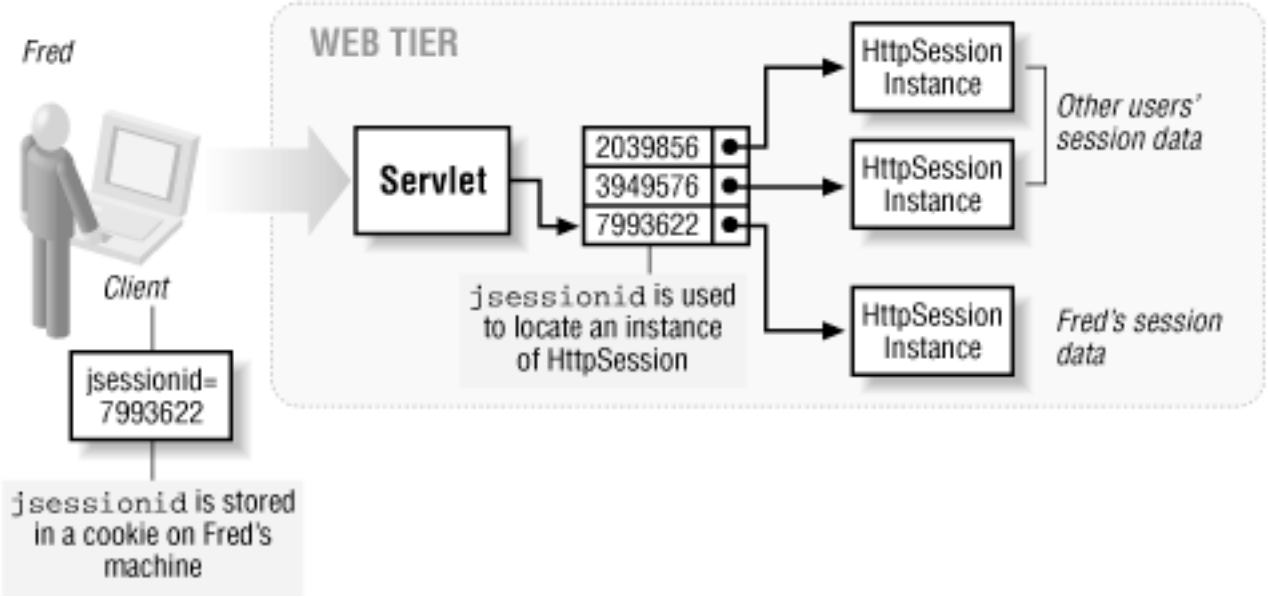
  Set-Cookie: JSESSIONID=0A83F8A1180B3577DC06CA96B128E33E; Path=/; HttpOnly

- Pri svakom sledećem zahtevu, browser će sam poslati cookies-e koje mu je server zakačio.

  Cookie: JSESSIONID=0A83F8A1180B3577DC06CA96B128E33E

Praćenje sesije

- Za sesiju možemo vezivati željene objekte, kojima može pristupiti samo vlasnik te sesije.
- Objekte možemo dodavati u HttpSession objekat metodom:
  - setAttribute(String s, Objec o)
- I kasnije im pristupiti metodom:
  - getAttribute(String s)



Praćenje sesije

- Korisniku možemo zkačiti i novi cookie, korišćenjem metode na instanci klase HttpServletResponse:
  - addCookie(Cookie cookie)
- Dodati cookie ili atributi će nam biti dostupni prilikom sledećeg zahteva.
- Zakačene cookies-e dobijamo pozivom metode na instance klase HttpServletRequest:
  - getCookies()